1 What Is an Embedded System

The word *system* comes from physics: a system is a set of elements inside a boundary. Everything inside the boundary is the system; everything outside is its *environment*. To describe an embedded system, the first step is therefore to decide what is inside the boundary (components, functions) and what is outside (the physical world it observes and controls).

There is no single official definition of an embedded system. Three common definitions stress different aspects, and together they give a complete picture.

View	Definition	Key idea
Hardware and software	Hardware plus software (*firmware*) built into it	The software cannot be written without knowing the hardware
Dedicated computer	A system based on a microcontroller or microprocessor that performs one specific task	Hardware and software are designed together for one job
Cyber-physical system	Computer hardware, software, and physical parts that perform a dedicated function	The computer interacts with the physical world

The third view explains what an embedded system usually does. *Sensors* measure physical quantities such as temperature, speed, or pressure. The processor runs the firmware and computes a response. *Actuators* turn that response into a physical action, such as moving a motor, opening a valve, or changing a display. Many embedded systems are also connected to other systems through a network.

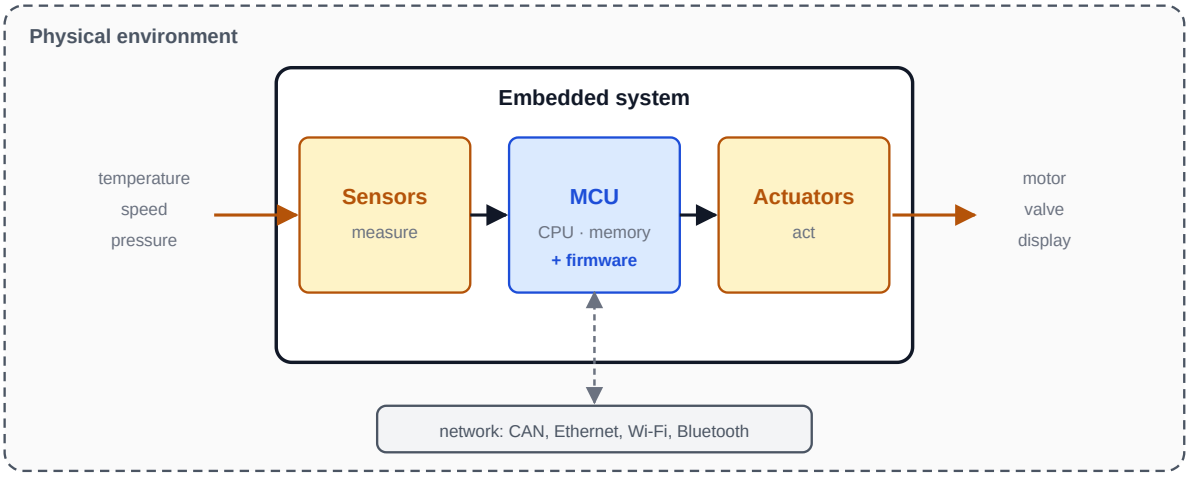


Figure 0.1: *An embedded system inside its environment. Sensors bring physical quantities in, the microcontroller runs the firmware, and actuators act on the world. A network may connect it to other systems.*

1.1 Classes of Embedded Systems

Embedded systems are used in very different fields, so they are classified in several ways. The three most useful classes are based on autonomy, timing, and connectivity. They are not exclusive: a car's brake controller is both real-time and networked.

Class	Main property	Example
Standalone	Works alone, without a network	Early washing machines, microwave ovens, thermostats
Real-time	Must complete each task before a deadline	Electronic brake control, drone stabilisation
Networked	Exchanges data with other nodes over a network or field bus	Engine, brake, and lighting controllers on a car's CAN bus

1.2 Embedded Versus General-Purpose Computers

A general-purpose computer, such as a laptop, runs any program the user installs, supports many users and tasks, and has plenty of memory and power. An embedded system runs one fixed job and works under strict limits.

Aspect	Embedded system	General-purpose computer
Workload	One fixed task	Anything the user installs
Resources	KB to a few MB of RAM and Flash	GB of RAM, TB of disk
Timing	Often hard deadlines	Best effort
User model	No interactive user, no shell	Multi-user, multi-tasking
Power and cost	Milliwatts, cents to a few euros	Tens to hundreds of watts, hundreds of euros

These limits change how software is written. Embedded software cannot rely on an operating system to create a process, assign an address space, discover devices, or provide standard input and output. It is built for one known processor and one known memory map. The hardware details are visible and cannot be ignored, but this does not make the software simple: the developer must handle many tasks that a desktop OS normally hides.

The border between the two worlds is becoming less sharp. The same processor families, especially ARM, now range from tiny sensor nodes to smartphones, laptops, and cloud servers.

1.3 A Short History

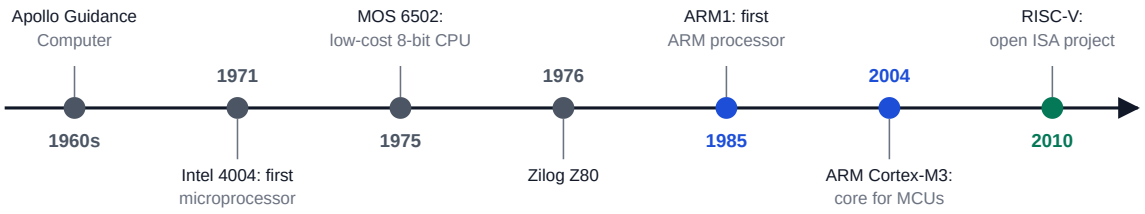


Figure 0.2: *Milestones of embedded computing, from the Apollo Guidance Computer to the open RISC-V instruction set (not to scale).*

The Apollo Guidance Computer, built in the 1960s for NASA's Moon missions, is often cited as one of the first true

embedded systems: a computer designed for one task and built into a vehicle. The first single-chip microprocessor, the Intel 4004 (1971), was designed by the Italian engineer Federico Faggin. Low-cost 8-bit processors such as the MOS 6502 and the Zilog Z80 followed in the 1970s and brought programmable control into consumer products.

Today the embedded market is dominated by ARM, whose low-power cores range from microcontrollers to servers. The Cortex-M3, introduced in 2004, is one of ARM's first cores designed specifically for microcontrollers. RISC-V is a newer alternative: its instruction set architecture (ISA) is open, so anyone can build compatible processors without paying licence fees.

Embedded technologies also have a long lifecycle. A satellite or an industrial machine may use the same processor for ten or twenty years, because replacing it is impossible or too expensive. Reliability and long-term support often matter more than performance.

2 Design Constraints of Embedded Systems

Embedded systems are designed under constraints that rarely matter on a desktop. Each one shapes both the hardware and the software.

Constraint	Why it matters	Example
Real-time	Results must arrive before a deadline	A brake controller must react within a few milliseconds
Size and weight	Extra mass costs energy and space	A heavier controller shortens a drone's flight time
Cost	Small extra costs multiply over millions of units	A 50-cent security chip rejected for a car because of the total cost
Energy	Many devices run on batteries	A wearable or remote sensor must last months on one battery
Reliability and safety	Failures can damage machines or hurt people	Medical devices and robots must *fail safe*
Development time	Low-level code is slow to write and test	Hardware debugging and testing in all operating conditions

Fail safe means that when a fault is detected, the system moves to a controlled and predictable state instead of acting in an uncontrolled way. In a safety-critical system, every failure scenario must be considered at design time.

2.1 Real-Time Means Bounded Response

Real-time does not mean fast. A real-time system guarantees a *bounded and predictable* response time: every reaction to an event must complete before its deadline. A correct embedded system must therefore meet three kinds of requirements.

Requirement	Example
Functional	Sample a sensor correctly
Temporal	Sample it every 1 ms, before the deadline
Robustness	Continue safely when one input is missing

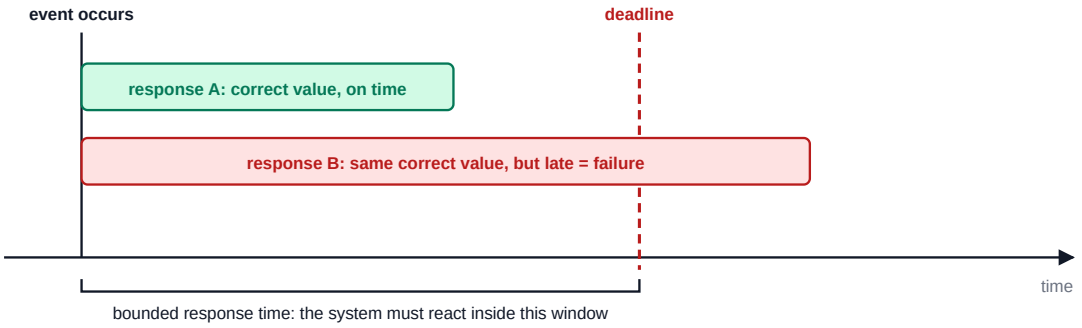


Figure 0.3: *Two responses to the same event. Both compute the correct value, but only the first arrives before the deadline; the second is a failure.*

In a *hard* real-time system, missing a deadline is a failure of the system, as in a braking controller. In a *soft* real-time system, a late result is still useful but lowers the quality of service, as in video playback.

> **CORRECT VALUE, WRONG TIME**
>
> A result computed after its deadline can be a failure even if its value is correct. A fast system is not necessarily a real-time system, because average speed says nothing about the worst case.

3 Inside the Hardware: Processor, Memory, and Buses

A computer has four basic parts: the central processing unit (CPU), memory, input units, and output units. An embedded developer works much closer to these parts than an application developer does, so a basic model of each is needed.

3.1 The CPU and the Pipeline

A CPU repeats the same cycle forever: it *fetches* an instruction from memory, *decodes* it to find out what to do, and *executes* it. Inside the CPU, the *control unit* drives this cycle, the *arithmetic logic unit* (ALU) performs arithmetic and logic operations, and *registers* hold the values being processed. Registers are the fastest storage in the system, and a few of them have special roles: the program counter, the stack pointer, and the status register.

Modern CPUs overlap these steps with a *pipeline*. Each step is handled by a different part of the hardware, so while one instruction executes, the next one is decoded and a third one is fetched, like cars in a car wash with one worker per station.

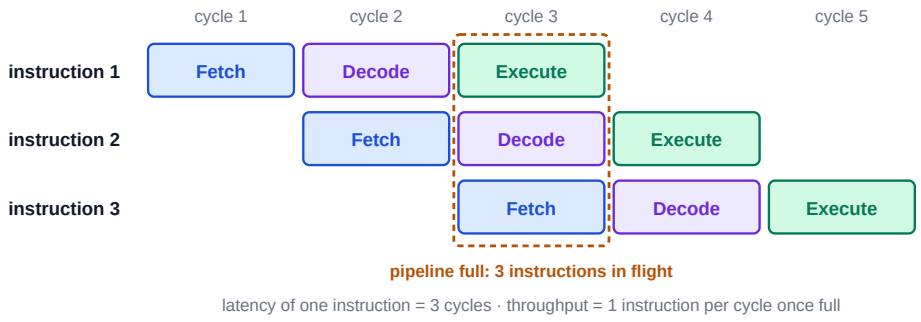


Figure 0.4: *A three-stage pipeline. From cycle 3 on, three instructions are in flight and one instruction completes every cycle.*

With n stages of duration t, a single instruction still needs $n \cdot t$ (its *latency* does not change), but once the pipeline is full one instruction completes every t. A sequence of m instructions takes

$$T_{total} = (n + m - 1) \cdot t$$

so pipelining increases *throughput*, not the speed of each instruction. The Cortex-M3 uses a three-stage pipeline of this kind.

3.2 Von Neumann and Harvard Architectures

CPU architectures differ in how they reach instructions and data. In the *von Neumann* architecture, instructions and data share one memory and one bus. It is simple and flexible, but the shared bus limits how fast data can move (the *von Neumann bottleneck*). In the *Harvard* architecture, instructions and data use separate memories and buses, so the CPU can fetch an instruction and access data at the same time.

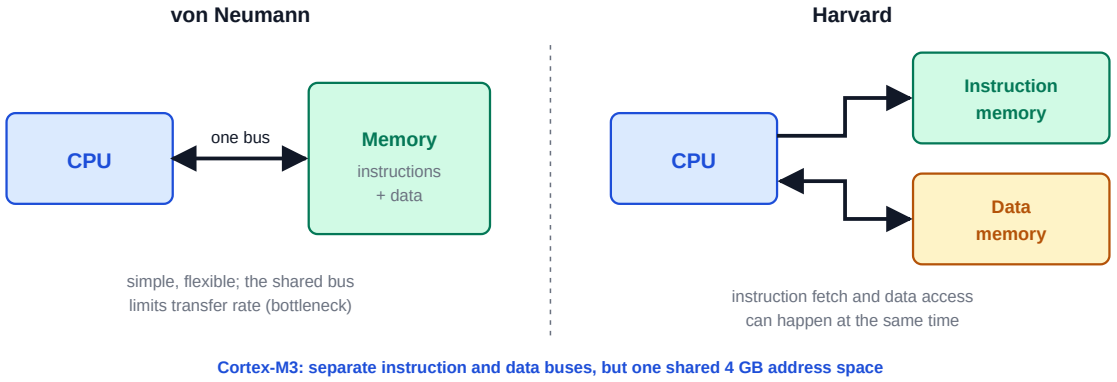


Figure 0.5: *von Neumann (one memory, one bus) and Harvard (separate instruction and data paths). The Cortex-M3 mixes the two: separate buses, one address space.*

Real processors often mix the two models. The Cortex-M3 has separate buses for instruction fetches and data accesses, which gives Harvard-like performance, but code and data share one 4 GB address space, as in von Neumann.

3.3 RISC and the Load/Store Model

ARM processors follow the RISC (Reduced Instruction Set Computer) philosophy: a small set of simple instructions, each executed quickly, combined with good compilers. A key RISC rule is the *load/store* model: arithmetic and logic work only on registers, so a value must be loaded from memory into a register, processed, and stored back. CISC (Complex Instruction Set Computer) processors, such as Intel x86, can instead operate directly on memory. Compilers hide this difference in C, but it becomes visible in assembly and in operating-system code.

3.4 Memory Hierarchy and Memory Technologies

Memories differ in speed, size, and cost, so every computer arranges them in a hierarchy. Small and fast memories sit close to the CPU; large and slow memories sit further away.

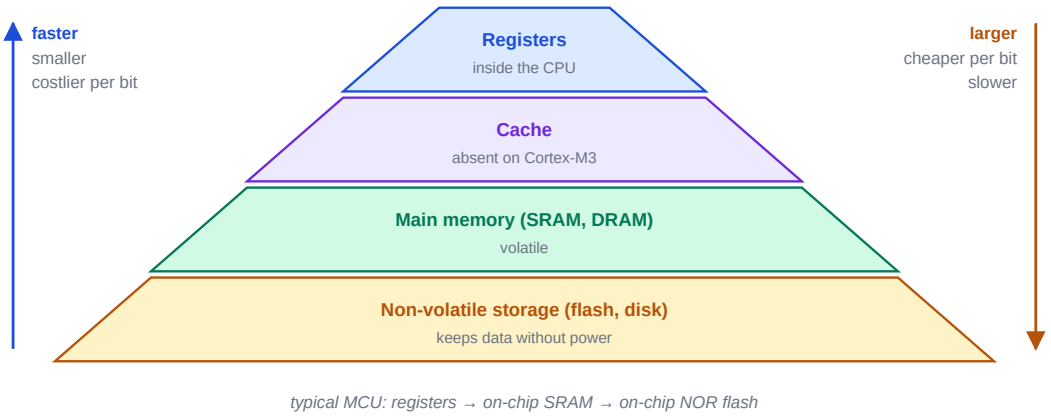


Figure 0.6: *The memory hierarchy. Going down, memories become larger and cheaper per bit but slower.*

Technology	Properties	Typical use
SRAM (static RAM)	Fast, no refresh needed, expensive, low density	On-chip RAM of a microcontroller, caches
DRAM (dynamic RAM)	Needs periodic refresh, cheap, high density	Main memory of PCs and large boards
NOR flash	Non-volatile, fast random reads, slow writes; code can run from it	Firmware and boot code
NAND flash	Non-volatile, accessed in blocks, dense and cheap	SD cards, USB drives, SSDs; not for direct execution

A *cache* is a small, fast memory that automatically keeps recently used data close to the CPU. It improves average speed, but its access time depends on what happened before, so the same code can take different times on different runs. This makes timing hard to predict, which is why many real-time systems disable caches or configure them carefully. The Cortex-M3 core has no cache at all.

3.5 Buses and Peripherals

Textbooks show a computer as a CPU and memories joined by a single address, data, and control bus. Real embedded boards have several buses: internal buses between the CPU and on-chip memory, buses to off-chip memory, and serial buses such as SPI, I²C, UART, or CAN to talk to sensors and other chips. Each peripheral is programmed by reading and writing its control registers, following the timing rules in its datasheet. On a desktop the operating system hides all of this; in an embedded system the firmware handles it directly.

3.6 Microprocessors and Microcontrollers

The core of a microprocessor and the core of a microcontroller execute instructions in the same way. The difference is what surrounds the core.

A *microprocessor* is mainly the CPU. It needs external RAM, external storage, and external peripheral chips to form a working computer. At power-on it runs a small firmware stored in a boot memory. This firmware starts a *boot loader*, which copies the operating system from storage (a disk or an SD card) into RAM and jumps to it.

A *microcontroller* (MCU) puts the CPU core, RAM, Flash, and peripherals (timers, converters, serial interfaces) on a single chip. Its internal Flash is mapped at the address the CPU reads after reset, so the firmware starts running directly from Flash, without a boot loader.

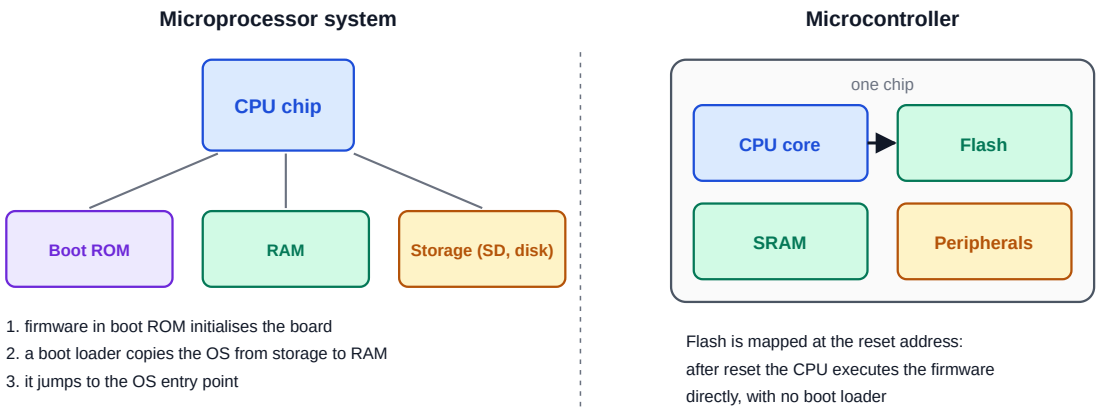


Figure 0.7: *A microprocessor needs external memories and a boot loader; a microcontroller integrates everything and executes its firmware straight from Flash after reset.*

4 Operating Systems

An operating system (OS) has several classic definitions, each stressing a different role.

Definition	Emphasis
Intermediary	A middle layer between hardware and applications that hides hardware details
Resource manager	Software that shares CPU, memory, I/O devices, and storage among applications
Master control program	The first software loaded at power-on; its core, the *kernel*, stays in memory

ROLE OF AN OPERATING SYSTEM

An OS mediates between application software and hardware. It gives controlled access to processors, memory, devices, and time, and it decides how competing work is scheduled.

The OS has two main purposes. First, it makes the hardware usable: without an OS, every application would have to initialise the CPU, program timers, share communication links, and handle interrupts on its own. Second, it manages resources that are limited and shared, even on a small board: CPU time (which task runs now), memory (which code and data occupy RAM), devices (who may use a timer or read a received byte), and events (how an interrupt becomes work for the right task).

Internally, an OS can be seen as a set of *managers*, one per resource, placed above a layer of hardware-specific services. Applications never call the managers directly: they go through a single *system-call interface*.

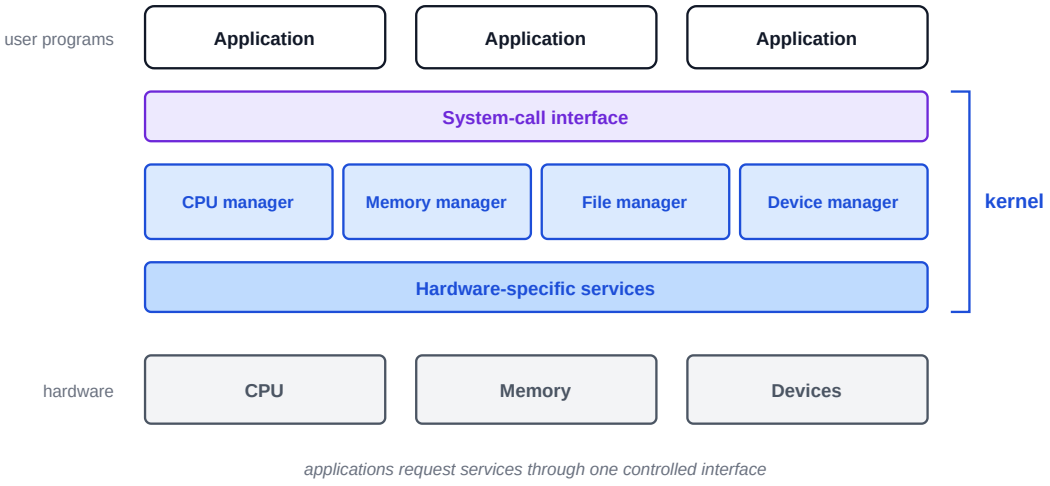


Figure 0.8: *The structure of an operating system. Applications use the system-call interface; the kernel's managers coordinate the hardware through hardware-specific services.*

4.1 Operating System Services

The services of a general-purpose OS fall into three groups.

Group	Services
For users	User interface (command line, graphical, touch, batch); program execution and termination

Group	Services
For programs	Uniform I/O operations through device drivers; file-system operations; communication between processes
For safe sharing	Resource allocation; logging and accounting; protection and security; error detection and handling

An embedded OS usually offers only a small part of this list. A microcontroller OS may provide only task scheduling, synchronisation, and time services, while the application keeps its own device drivers.

> **THE OS BOUNDARY IS A DESIGN CHOICE**
>
> What belongs to the OS is not a fixed feature list. It depends on what the product needs.

4.2 Privilege Levels, Traps, and System Calls

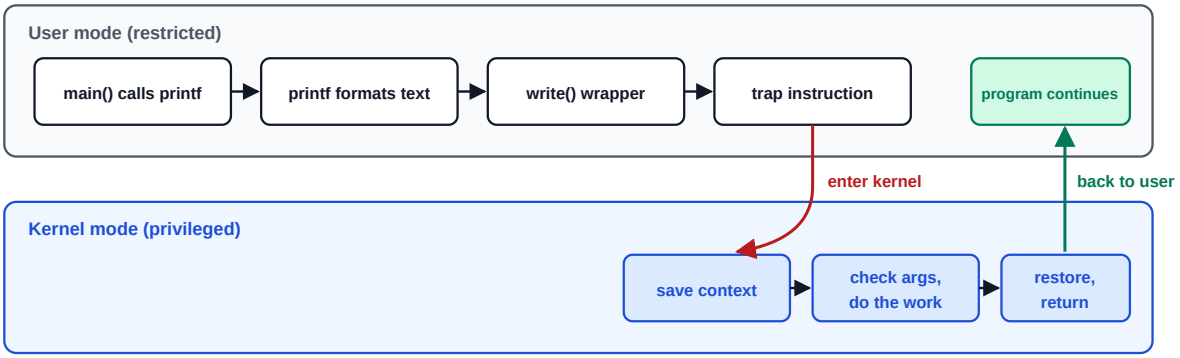
To protect itself and the hardware, an OS relies on CPU *execution modes*. Application code runs in *user mode* (restricted): it cannot program I/O controllers, change memory-protection settings, or disable interrupts. The kernel runs in *kernel mode* (privileged), where everything is allowed.

When an application needs a privileged operation, it makes a *system call*. A system call is implemented with a *trap*, a special instruction that moves execution from user mode to kernel mode in a controlled way. The CPU switches mode, saves the context of the program (for example the program counter and some registers), and jumps to a fixed kernel entry point. The kernel checks the arguments, performs the service, prepares a return value, restores the context, and returns to user mode.

```
#include <unistd.h>
#include <string.h>

int main( void )
{
    const char *msg = "Greetings\n";
    ssize_t n = write( 1, msg, strlen( msg ) );  /* 1 = standard output */
    return ( n < 0 ) ? 1 : 0;
}
```

Programs rarely execute the trap instruction themselves. They call library functions such as `printf`, which formats the text in user mode and then calls `write`, a thin *wrapper* that places the arguments where the kernel expects them and executes the trap. The return value tells the program whether the operation succeeded.



the same shape on Cortex-M: svc enters a privileged handler, exception return goes back

Figure 0.9: *A system call. The program reaches the trap through the C library, the kernel does the work in privileged mode, and control returns to the program in user mode.*

The Cortex-M3 implements the same mechanism with the `svc` instruction and the privileged Handler mode, as described in the chapter on the ARM Cortex-M3 architecture.

4.3 From Source Code to a Running OS

A program becomes executable in three steps. The *compiler* turns each source file into a *relocatable object file*, whose final addresses are not fixed yet. The *linker* combines the object files and libraries into one executable image (an ELF file), resolves the references between them, and assigns final addresses. Finally, the image must be placed in memory and started: on a PC this is the job of a *loader*, which can also relocate the program to a different address.

On a microcontroller the OS image is written into Flash with an external programmer during development, and the CPU runs it from its reset address. Applications can be added to such a system in two ways. With *fixed tasks*, the tasks are compiled and linked together with the OS into one image, and no loader is needed. With *programmable tasks*, applications are separate images loaded after the OS starts; this allows updates without rebuilding the OS, at the cost of a loader, memory management, and security checks. Most small embedded systems use fixed tasks.

5 Kernel Architectures

Operating systems differ in *where* their services run and *how strongly* they are isolated from each other and from applications.

Architecture	Where services run	Advantage	Cost	Examples
Flat	Everything in one image and one address space	Small and fast	Little or no isolation	FreeRTOS, MS-DOS
Monolithic	All services in one privileged kernel	Fast direct calls between services	Large trusted core; a bad driver can crash the kernel	Linux, UNIX
Microkernel	Minimal kernel; services as user-mode servers	Small trusted base, faults contained	IPC overhead, more interfaces	seL4, QNX
Hybrid	Mix of the above	Balance of speed and structure	Complexity	macOS (Mach + BSD), Linux modules

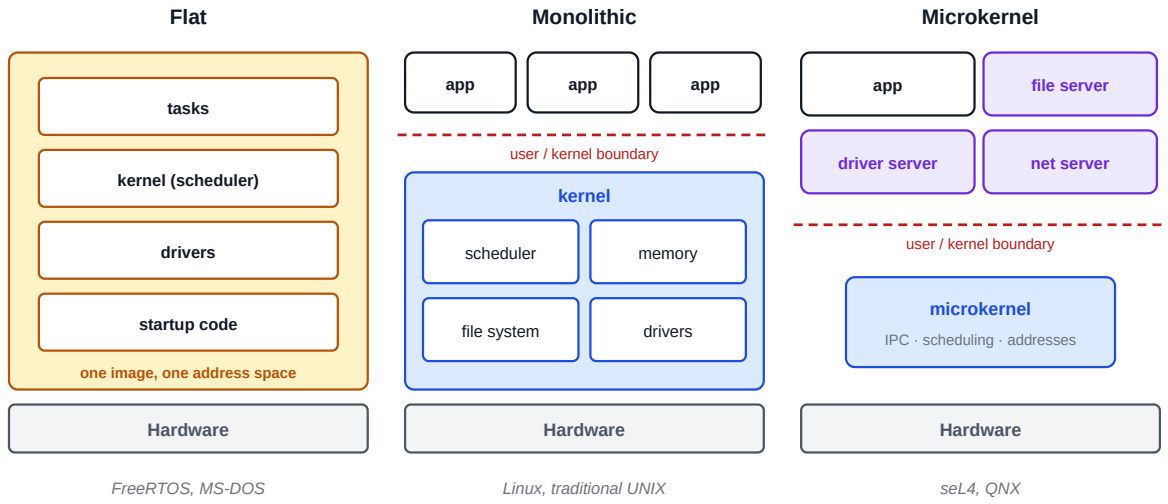


Figure 0.10: *Three kernel architectures. The red line is the hardware-enforced boundary between user and kernel mode; a flat system has none.*

In a *flat* design, startup code, kernel, drivers, and tasks are linked into one image and share one physical address space. OS services are ordinary functions that tasks call directly. No memory management unit and no privilege levels are needed, but any bug can corrupt any data.

A *monolithic* kernel separates user space from kernel space with hardware support (privilege levels and an MMU or MPU), so a faulty application cannot corrupt the kernel. Inside the kernel, however, scheduler, memory manager, file systems, and drivers share the same privileged space. Drivers are a large and error-prone part of the code, and a single faulty driver can crash the whole system.

A *microkernel* keeps only the basic mechanisms in kernel mode, typically scheduling, address-space management, and inter-process communication (IPC). File systems, drivers, and network stacks run as user-mode servers that exchange messages. A failed server can be restarted, and the small kernel is easier to verify.

The price is performance: a system call in a monolithic kernel needs two mode switches (user to kernel and back), while the same request in a microkernel typically needs four (client to kernel, kernel to server, server to kernel, kernel to client).

Most real systems are *hybrids*. Linux is monolithic but loads drivers as modules at run time; macOS combines the Mach microkernel with BSD components.

CHOOSING A KERNEL ARCHITECTURE

Each architecture is a trade-off between isolation, fault containment, overhead, and complexity. The useful question is not "which one is best?" but "what must this system guarantee?".

6 Embedded and Real-Time Operating Systems

Embedded OSes differ from desktop OSes for three reasons. They are *resource-constrained*: a few KB of RAM leave no room for a large kernel. They are *real-time*: correctness depends on when things happen. They usually have *no untrusted users*: one vendor ships one firmware image with a known set of tasks. For these reasons an embedded OS is selected and configured for one product, not for an unknown future workload.

General-purpose mechanism	Embedded replacement
Dynamic allocation	Static number of tasks and fixed-size buffers
General fairness policy	Simple priority-based scheduler
Layered device stack	Direct register access
Maximum throughput	Predictable execution time

SMALL IS NOT INCOMPLETE

A small OS is complete when it matches the responsibilities of the system.

A *real-time operating system* (RTOS) such as FreeRTOS provides ready-made versions of the mechanisms an embedded application needs: tasks with priorities, delays and timer-driven scheduling, queues, semaphores and mutexes, and communication from interrupts to tasks. By default FreeRTOS has no memory protection between tasks: like flat firmware, all tasks share one address space. Knowing the mechanisms underneath an RTOS makes its API much easier to use correctly.

6.1 Single Image, Single Failure Domain

Many microcontroller products ship one firmware image that contains startup code, drivers, OS, and application. Updating the image changes the whole system. This makes deployment simple, but it requires discipline: a bad pointer can damage any writable data, a blocked high-priority task can stop an important service, and recovery and diagnostics must be designed into the firmware from the start. Scheduling, synchronisation, and memory protection are the mechanisms that reduce these risks.

6.2 From Bare Metal to a Minimal OS

A minimal flat OS is built as a chain of mechanisms, where each one depends on the previous one. This chain is also the path followed by the rest of these notes, on an ARM Cortex-M3 microcontroller emulated by QEMU.

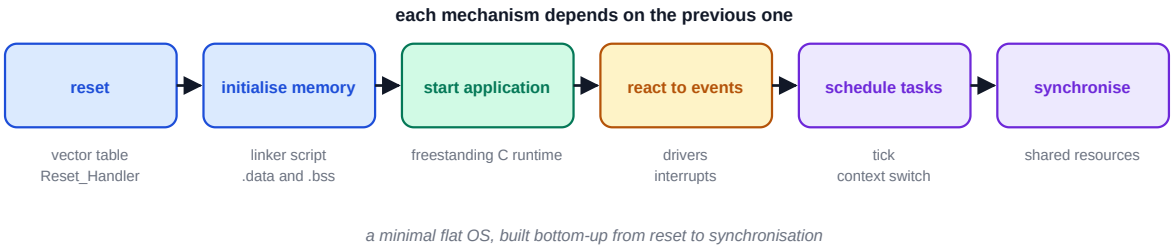


Figure 0.11: *The mechanisms of a minimal OS, from reset to synchronisation, with the main topics involved in each step.*

First the processor must start correctly after reset, which requires a vector table and a reset handler. Then memory must be prepared, as described by the linker script, so that C code can run. Once the application runs, it must react to events through device drivers and interrupts. With a periodic timer tick and context switching, the kernel can schedule several tasks, and finally it must synchronise their access to shared resources. Memory protection and system calls, built on the privilege levels of the Cortex-M3, add isolation on top of this chain.